

Quantization

* A lot of the very helpful images are from an article I read to help me with quantization, cited at the end.

Why Quantization?

- Models have billions of parameters - GPT3 is ~700GB.
- Makes models smaller → Get over the bottleneck in model size in deployment → Cheaper, faster, smaller processors. (Reduce size, increase speed)

Quantized models are faster because you lose precision in the weights (FP32 → Int8)

InfoWorld - “Quantization not only makes it possible to run a LLM on a single GPU, it allows you to run it on a CPU or on an edge device.”

What is Quantization?

Definition: Converting high-precision weights to lower-precision formats

Common precision formats:

- FP32 (32-bit float): 4 bytes → Standard training format
- INT8 (8-bit integer): 1 byte → Common quantization target
- INT4/FP4 (4-bit): 0.5 bytes → Aggressive quantization

Example: FP32: 3.14159265... → INT8: 3 (75% less memory)

Trade-off: Speed & Size ↔ Accuracy

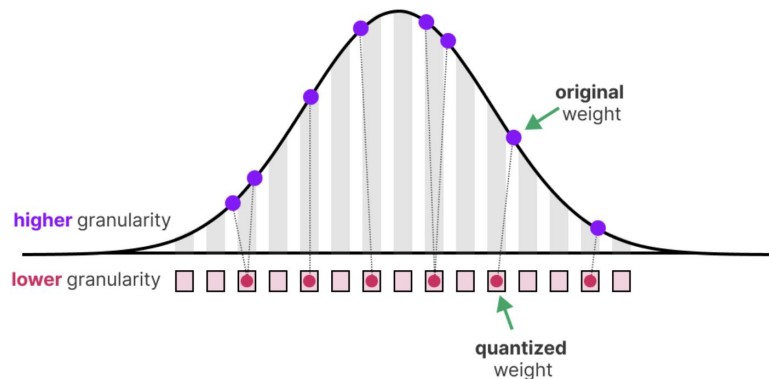
Types of Quantization

Granularity: Per-Tensor vs. Per-Channel

Per-Tensor: One scale factor for entire weight matrix

Per-Channel: Separate scale factor for each channel

→ Per-channel: Larger file but better accuracy



Types of Quantization

Timing: Post-Training (PTQ) vs. Quantization-Aware (QAT)

PTQ: Quantize after training (simpler, my approach)

QAT: Train with quantization in mind (more accurate)

→ PTQ with calibration can get close to QAT results

From RepQ-ViT Paper: “Thus, PTQ, which is free from retraining, is believed to be a more promising solution for low-cost and rapid deployment.”

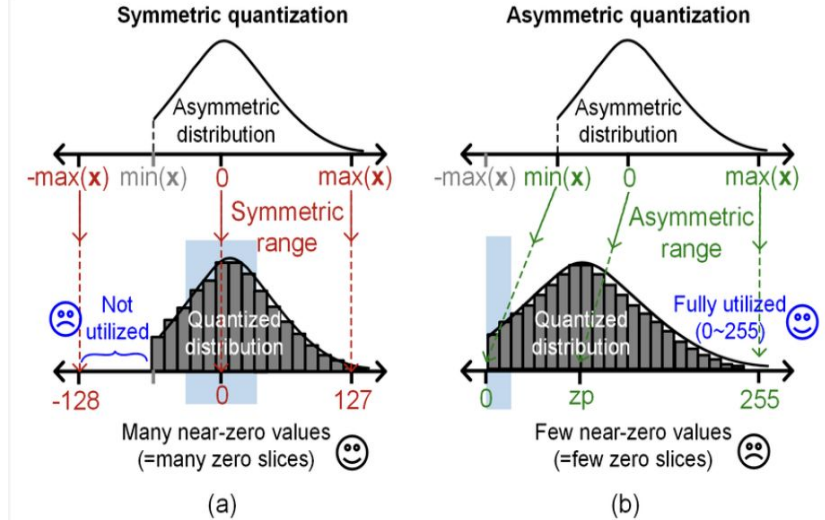
Types of Quantization Cont.

Range Mapping: Symmetric vs. Asymmetric

Symmetric: Zero maps to zero (-127 to 127 for INT8)

Asymmetric: Full range mapping (-128 to 127 for INT8)

→ Symmetric is simpler, asymmetric can be more accurate



Workflow

Testing on different architectures and datasets:

- Architectures: ViT, L1, L2
- Datasets: CHB-MIT, NICU, Private

Workflow: Iterate through state dictionary.

- Mixed Precision Check
- Tensor/Channel Check
- Quantization
- Save New Tensors to State Dictionary

```
class MixedPrecisionAnalyze():
    def should_quantize_tensor(self,
                              tensor: torch.Tensor,
                              layer_name: Optional[str] = None) -> bool:
        if layer_name is not None:
            return self._check_layer_rules(layer_name)
        return True

    def _check_layer_rules(self, layer_name: str) -> bool:
        """Apply layer-specific quantization rules."""

        layer_name_lower = layer_name.lower()

        # Generally avoid quantizing these layer types
        sensitive_layers = [
            'norm', 'embed',
            'head', 'cls']

        for sensitive in sensitive_layers:
            if sensitive in layer_name_lower:
                return False

        return True
```

My Type of Quantization

1. Mixed Precision Strategy

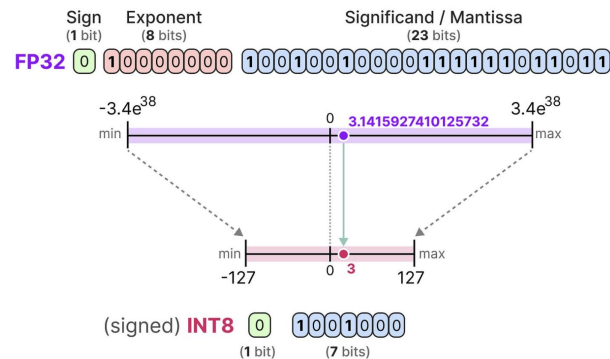
- Sensitive layers (norm, embed, head, cls) → Keep FP32
- All other layers → Quantize to INT8
- Balance between compression and accuracy

2. Symmetric INT8 Quantization

- Single scale factor per channel – Scale between - 128 to 127

3. Why INT8 over FP4?

- Better hardware support (NVIDIA TensorRT, Intel VNNI, ARM)
- Proven accuracy retention (<1% degradation)
- 4× memory reduction vs FP32



My Type of Quantization

Average accuracy change: **-0.05%**

Drop range: -0.69% to +0.23%

Models with $< 0.1\%$ change: 6/9 (67%)

Model size reduction: $\sim 75\%$

Model Layers	
Input Embedding	[FP32]
Norm Layer 1	[FP32]
Attention Weights	[INT8]
Feed-Forward Weights	[INT8]
Norm Layer 2	[FP32]
Classification Head	[FP32]

		Original Accuracy	Quantized Accuracy	Accuracy Drop
CHB-MIT	ViT	80.1	79.87	0.23
	L1	80.66	80.68	-0.02
	L2	76.49	76.38	0.11
NICU	ViT	98.54	98.52	0.02
	L1	98.43	98.52	-0.09
	L2	98.58	98.59	-0.01
Private	ViT	85.96	85.93	0.03
	L1	86.57	87.26	-0.69
	L2	89	88.83	0.17

How can accuracy increase?

1. Quantization Regularization Effect: Rounding improves generalization.
2. Lucky Rounding on Borderline Predictions: Can push to correct class.
3. Statistical Noise

Quantization Limitations

Accuracy Degradation: From losing precision, this usually happens.

Calibration Challenges: Need a dataset to calibrate but data is always a struggle in ML.

Hardware Limitations: Some CPU/GPU's do not support Int8.

When Quantization Struggles:

- Very small models (already compact)
- Tasks requiring high numerical precision
- Dynamic range in activations

Sources

- [1] S. Ozturk, “Understanding LLM Quantization: A Deep Dive into Performance Optimization,” TheMeanSquare – Medium, Jan. 9, 2025. [Online]. Available: <https://themean-square.medium.com/understanding-llm-quantization-a-deep-dive-into-performance-optimization-c27d63857faa>
- [2] M. Grootendorst, “A Visual Guide to Quantization,” MaartenGrootendorst.com — Blog, Jul. 22, 2024. [Online]. Available: <https://www.maartengrootendorst.com/blog/quantization/>
- [3] Z. Li, J. Xiao, L. Yang and Q. Gu, "RepQ-ViT: Scale Reparameterization for Post-Training Quantization of Vision Transformers," 2023 IEEE/CVF International Conference on Computer Vision (ICCV), Paris, France, 2023, pp. 17181-17190, doi: 10.1109/ICCV51070.2023.01580.
keywords: {Degradation;Computer vision;Quantization (signal);Codes;Computational modeling;Transformers;Hardware},